

INFORME TÉCNICO

Alta Disponibilidad y Sincronización de Servicios en Red Hat Enterprise Linux 10 (RHEL 10)

Prácticas: Rsync + Cron · Pacemaker + Corosync · Keepalived + VRRP

Autor	Víctor José De Peña Bernard
Matrícula	2023-0506
Institución	Instituto Tecnológico de Las Américas (ITLA)
Asignatura	Prácticas de Ciberseguridad / Administración de Servidores Linux
Sistema operativo	Red Hat Enterprise Linux 10 (RHEL 10)
Fecha	Julio de 2026

1. Introducción

El presente informe documenta la implementación práctica de tres laboratorios enfocados en la sincronización, automatización y alta disponibilidad (HA, High Availability) de servicios sobre el sistema operativo Red Hat Enterprise Linux 10 (RHEL 10). El objetivo de estas prácticas es familiarizar al estudiante con herramientas de administración de sistemas ampliamente utilizadas en entornos empresariales para garantizar la continuidad operativa, la replicación de datos y la resiliencia ante fallos de servidores.

Se desarrollaron tres escenarios independientes pero complementarios: (1) sincronización automática de archivos entre dos servidores mediante Rsync y Cron, (2) un clúster de alta disponibilidad genérico gestionado con Pacemaker y Corosync, y (3) un clúster de alta disponibilidad específico para un servicio web HTTP mediante Keepalived y el protocolo VRRP. En conjunto, estas prácticas abarcan tanto la replicación a nivel de archivos como la disponibilidad a nivel de servicio e IP.

2. Objetivos

2.1 Objetivo general

Implementar y validar mecanismos de sincronización de datos, automatización de tareas y alta disponibilidad de servicios sobre servidores RHEL 10, empleando herramientas estándar de la industria.

2.2 Objetivos específicos

- Configurar la sincronización automática de archivos entre un servidor principal y uno secundario utilizando Rsync sobre SSH.
- Automatizar la ejecución periódica de tareas mediante el demonio Cron.
- Diseñar e implementar un clúster de alta disponibilidad de dos nodos utilizando Pacemaker y Corosync.
- Configurar una dirección IP virtual (Floating IP) que garantice la continuidad del servicio ante la caída de un nodo.
- Implementar alta disponibilidad para un servicio web Apache HTTP Server mediante Keepalived y el protocolo VRRP.
- Validar experimentalmente el comportamiento de failover en cada escenario.

3. Entorno y requisitos generales

Los tres laboratorios comparten un mismo entorno base de dos servidores virtualizados con RHEL 10, comunicados en red mediante interfaces en modo Bridge. Los requisitos generales son los siguientes:

- Dos servidores con Red Hat Enterprise Linux 10 (RHEL 10) instalado.
- Acceso con privilegios administrativos (sudo / root).

- Comunicación de red bidireccional entre ambos nodos.
- Acceso SSH configurado entre los servidores.
- Interfaces de red en modo Bridge, para permitir direccionamiento visible en la red local.

Direccionamiento IP utilizado a lo largo de las prácticas:

Nodo 1 (Servidor Master)	172.27.222.108
Nodo 2 (Servidor Backup)	172.27.218.146 (usuario hacku)
IP Flotante — Práctica 2 (Pacemaker)	172.27.218.150 /16
IP Virtual — Práctica 3 (Keepalived / VRRP)	172.27.218.100 /16

4. Práctica 1 — Sincronización de carpetas con Rsync

4.1 Descripción

Se configuró un mecanismo de sincronización unidireccional entre un servidor principal y un servidor secundario mediante Rsync sobre SSH, automatizado a través de un script de Bash ejecutado periódicamente por Cron. El objetivo es mantener replicado en tiempo casi real el contenido de un directorio de trabajo hacia un servidor remoto de respaldo.

4.2 Actividades realizadas

- Creación de una carpeta de trabajo en el servidor principal.
- Generación de 100 archivos de prueba mediante el comando touch.
- Sincronización inicial del contenido hacia el servidor remoto utilizando rsync.
- Creación de un script Bash (sincronizar.sh) para automatizar la sincronización.
- Asignación de permisos de ejecución al script mediante chmod +x.
- Ejecución manual de prueba para validar la ausencia de errores.
- Configuración de una tarea programada en Crontab para ejecutarse cada minuto.
- Creación de un archivo adicional para validar el funcionamiento automático del Cron.
- Verificación de la sincronización desde el servidor secundario.

4.3 Comandos y configuración

Pasos de configuración del script y de la tarea programada en el servidor principal:

```
# Crear y editar el script de sincronización automatizada
$ nano ~/sincronizar.sh
```

```
# Asignar permisos de ejecución al script (paso crítico para el Cron)
$ chmod +x ~/sincronizar.sh

# Ejecución de prueba manual para validar que no existan errores
$ /bin/bash /home/redhat/sincronizar.sh

# Configurar la tarea programada en el demonio Cron
$ crontab -e

# Crear un archivo de prueba adicional para validar el Cron
$ touch ~/sincronizacion_practica/archivo_final.txt
```

Contenido del script de sincronización (~sincronizar.sh):

```
#!/bin/bash
# Script de sincronización automática - Práctica 1

ORIGEN="$HOME/sincronizacion_practica/"
DESTINO="hacku@172.27.218.146:/home/hacku/sincronizacion_remota/"

# Uso de la ruta absoluta del binario rsync para entornos Cron seguros
/usr/bin/rsync -avz "$ORIGEN" "$DESTINO"
```

Línea de tarea programada agregada en el Crontab (crontab -e):

```
* * * * * /bin/bash /home/redhat/sincronizar.sh > /dev/null 2>&1
```

Los cinco asteriscos (* * * * *) indican al sistema operativo que ejecute el script de forma estricta cada minuto, de manera indefinida.

4.4 Verificación final

Desde el servidor secundario, se ejecutó el siguiente comando de listado tras esperar un minuto:

```
$ ls -la /home/hacku/sincronizacion_remota/
```

Como resultado se observaron los 100 archivos base y el archivo `archivo_final.txt` sincronizados correctamente, con sus marcas de tiempo actualizadas, confirmando el correcto funcionamiento de la replicación automática vía Rsync + Cron.

4.5 Herramientas utilizadas

Herramienta	Función en la práctica
RHEL 10	Sistema operativo base de ambos servidores
Rsync	Sincronización eficiente de archivos vía SSH
OpenSSH	Canal seguro de transporte para Rsync
Bash	Automatización mediante script de sincronización
Cron	Programación de la ejecución periódica del script

5. Práctica 2 — Clúster de Alta Disponibilidad con Pacemaker y Corosync

5.1 Descripción

Se implementó un clúster de alta disponibilidad compuesto por dos nodos utilizando Pacemaker (gestor de recursos del clúster) y Corosync (motor de mensajería e infraestructura de clúster), configurando una dirección IP virtual (Floating IP) que garantiza la continuidad del servicio ante la caída de cualquiera de los nodos.

5.2 Topología del entorno

Nodo 1	172.27.222.108
Nodo 2	172.27.218.146
IP Flotante (Virtual IP)	172.27.218.150 (máscara /16)

5.3 Fase 1 — Limpieza de configuraciones previas (en ambos nodos)

Comandos utilizados para reiniciar el laboratorio desde cero y evitar conflictos residuales:

```
sudo pcs cluster stop
sudo pcs cluster destroy
sudo dnf remove -y pacemaker pcs corosync resource-agents
sudo rm -rf /etc/corosync/* /var/lib/pcsd/* /var/lib/pacemaker/*
```

5.4 Fase 2 — Preparación e instalación del entorno (en ambos nodos)

1) Configuración de la resolución de nombres local (/etc/hosts):

```
sudo bash -c 'cat <<EOF >> /etc/hosts
172.27.222.108  nodo1
172.27.218.146  nodo2
EOF'
```

2) Habilidad del repositorio de Alta Disponibilidad para RHEL 10:

```
sudo subscription-manager repos --enable=rhel-10-for-x86_64-highavailability-rpms
sudo dnf clean all
sudo dnf makecache
```

3) Instalación de los paquetes del clúster y herramientas del sistema:

```
sudo dnf install -y pacemaker pcs corosync resource-agents psmisc
```

4) Activación e inicio del demonio de configuración de PCS:

```
sudo systemctl enable --now pcsd
```

5) Configuración de la contraseña del usuario administrador del clúster (hacluster), estableciendo la misma contraseña en ambos servidores:

```
sudo passwd hacluster
```

6) Configuración del firewall para permitir el tráfico de alta disponibilidad:

```
sudo firewall-cmd --permanent --add-service=high-availability
sudo firewall-cmd --reload
```

5.5 Fase 3 — Creación y configuración del clúster (solo en el Nodo 1)

1) Autenticación entre nodos con el usuario hacluster:

```
sudo pcs host auth nodo1 nodo2
```

2) Generación de la configuración de Corosync y estructuración del clúster:

```
sudo pcs cluster setup mi_cluster nodo1 nodo2 --force
```

3) Inicio simultáneo de los servicios del clúster en todos los nodos:

```
sudo pcs cluster start --all
```

4) Habilitación del arranque automático del clúster con el sistema operativo:

```
sudo pcs cluster enable --all
```

5) Desactivación de STONITH (obligatorio en laboratorios de 2 nodos sin hardware de fencing):

```
sudo pcs property set stonith-enabled=false
```

6) Configuración de la política de cuórum para ignorar la pérdida de la mayoría absoluta:

```
sudo pcs property set no-quorum-policy=ignore
```

7) Creación del recurso de red para la IP Flotante en el segmento /16:

```
sudo pcs resource create IP_Flotante ocf:heartbeat:IPaddr2 \
ip=172.27.218.150 cidr_netmask=16 op monitor interval=30s
```

5.6 Fase 4 — Monitoreo y pruebas de conmutación por error (failover)

1) Verificación del estado en tiempo real del clúster (desde cualquier nodo):

```
sudo pcs status
```

2) Inicio de una prueba de conectividad continua desde una terminal Windows CMD:

```
ping 172.27.218.150 -t
```

3) Simulación de caída del Nodo 1 — la IP flotante migra automáticamente al Nodo 2:

```
sudo pcs cluster stop nodo1
```

4) Recuperación del Nodo 1 y reintegración al clúster:

```
sudo pcs cluster start nodo1
```

5) Simulación de caída del Nodo 2 — alternancia de servicios, la IP regresa al Nodo 1:

```
sudo pcs cluster stop nodo2
```

Durante las pruebas de failover se comprobó que, ante la caída de cualquiera de los dos nodos, la IP flotante 172.27.218.150 migra automáticamente hacia el nodo disponible, manteniendo el ping continuo activo con una interrupción mínima, lo que valida el correcto funcionamiento del clúster de alta disponibilidad.

5.7 Herramientas utilizadas

Herramienta	Función en la práctica
RHEL 10	Sistema operativo base de ambos nodos del clúster
Pacemaker	Gestor de recursos y orquestador del clúster de HA
Corosync	Motor de mensajería e infraestructura de clúster
pcs	Interfaz de línea de comandos para administrar el clúster
Floating IP	Recurso de IP virtual gestionado por el clúster

6. Práctica 3 — Clúster de Alta Disponibilidad HTTP con Keepalived

6.1 Descripción

Se implementó un servicio web altamente disponible utilizando Apache HTTP Server y Keepalived. Cada servidor aloja una página HTML que identifica al nodo correspondiente (Server 1 y Server 2), mientras que Keepalived administra una dirección IP virtual (VIP) mediante el protocolo VRRP para ofrecer acceso transparente al servicio, independientemente de cuál de los dos servidores esté activo.

6.2 Topología del entorno

Servidor Master	172.27.222.108 (interfaz ens160)
Servidor Backup	172.27.218.146 (interfaz ens224)
IP Virtual (VIP)	172.27.218.100 /16

6.3 Configuración del Servidor Master (172.27.222.108)

a) Instalación de paquetes y creación de directorios:

```
sudo dnf install -y keepalived httpd
sudo mkdir -p /etc/keepalived /var/www/holamundo
```

b) Configuración de Keepalived (/etc/keepalived/keepalived.conf):

```
global_defs {
    router_id rhel10_master
}
vrrp_script check_httpd {
    script "pidof httpd"
    interval 2
    weight 2
}
vrrp_instance VI_1 {
    state MASTER
    interface ens160
    virtual_router_id 51
    priority 101
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass SecHttp123
    }
    virtual_ipaddress {
        172.27.218.100/16
    }
    track_script {
        check_httpd
    }
}
```

c) Configuración de la página web, firewall y servicios:

```
echo "<h1>Hola desde SERVER 1 (Master - 172.27.222.108)</h1>" | \
  sudo tee /var/www/holamundo/index.html
sudo chmod 644 /var/www/holamundo/index.html
sudo rm -f /etc/httpd/conf.d/welcome.conf
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-protocol=vrrp --permanent
sudo firewall-cmd --reload
sudo systemctl enable --now httpd
sudo systemctl enable --now keepalived
```

6.4 Configuración del Servidor Backup (172.27.218.146)

a) Instalación de paquetes y creación de directorios:

```
sudo dnf install -y keepalived httpd
sudo mkdir -p /etc/keepalived /var/www/holamundo
```

b) Configuración de Keepalived (/etc/keepalived/keepalived.conf):

```
global_defs {
    router_id rhel10_backup
}
vrrp_script check_httpd {
    script "pidof httpd"
    interval 2
    weight 2
}
vrrp_instance VI_1 {
    state BACKUP
    interface ens224
    virtual_router_id 51
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass SecHttp123
    }
    virtual_ipaddress {
        172.27.218.100/16
    }
    track_script {
        check_httpd
    }
}
```

c) Configuración de la página web, firewall y servicios:

```
echo "<h1>Hola desde SERVER 2 (Backup - 172.27.218.146)</h1>" | \
  sudo tee /var/www/holamundo/index.html
```

```

sudo chmod 644 /var/www/holamundo/index.html
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-protocol=vrrp --permanent
sudo firewall-cmd --reload
sudo systemctl enable --now httpd
sudo systemctl enable --now keepalived

```

6.5 Comandos de diagnóstico y pruebas

Verificación de que la IP virtual está activa en la interfaz de red:

```
ip addr show
```

Prueba local de la respuesta del servidor Apache:

```
curl http://localhost
```

Simulación de caída del servidor Master, deteniendo el servicio Apache:

```
sudo systemctl stop httpd
```

Recuperación del estado inicial, reiniciando Apache en el Master:

```
sudo systemctl start httpd
```

Al detener Apache en el servidor Master, el script de verificación `check_httpd` detecta la ausencia del proceso y Keepalived reduce la prioridad del nodo, provocando que la IP virtual 172.27.218.100 migre automáticamente hacia el servidor Backup. Al acceder desde el navegador a dicha IP durante la falla, se visualiza la página correspondiente al Server 2, confirmando la continuidad del servicio web sin intervención manual.

6.6 Herramientas utilizadas

Herramienta	Función en la práctica
RHEL 10	Sistema operativo base de ambos servidores
Apache HTTP Server	Servicio web monitoreado por Keepalived
Keepalived	Gestión de la IP virtual y del failover del servicio
VRRP	Protocolo de redundancia de enrutador virtual utilizado por Keepalived

7. Conclusiones

Las tres prácticas desarrolladas permitieron abordar, de forma progresiva, distintos niveles de continuidad operativa en entornos Linux: desde la replicación de archivos a nivel de sistema de archivos (Rsync + Cron),

pasando por la gestión genérica de recursos de clúster (Pacemaker + Corosync), hasta la alta disponibilidad orientada a un servicio específico mediante failover de IP virtual (Keepalived + VRRP).

La práctica de Rsync y Cron demostró ser una solución sencilla y de bajo costo operativo para mantener réplicas actualizadas de datos entre servidores, aunque su naturaleza por lotes (cada minuto) implica una ventana de posible desincronización, a diferencia de los mecanismos de failover en tiempo real evaluados en las prácticas 2 y 3.

El clúster con Pacemaker y Corosync evidenció ser una herramienta robusta y flexible para la gestión de recursos de alta disponibilidad genéricos, permitiendo administrar no solo direcciones IP sino potencialmente cualquier recurso o servicio adicional mediante agentes OCF. Por su parte, Keepalived con VRRP resultó una alternativa más liviana y específica, orientada a escenarios donde el objetivo principal es la disponibilidad de un servicio de red puntual, como un servidor web.

En ambos escenarios de alta disponibilidad (Pacemaker/Corosync y Keepalived) se validó experimentalmente que, ante la caída simulada de un nodo, el servicio y/o la IP virtual migran automáticamente hacia el nodo disponible, cumpliendo con el objetivo de continuidad operativa sin intervención manual, lo cual constituye una competencia fundamental en la administración de infraestructuras críticas de Linux.

8. Autor

Víctor José De Peña Bernard — Matrícula 2023-0506

Prácticas desarrolladas en Red Hat Enterprise Linux 10 (RHEL 10) para la asignatura de Administración de Servidores Linux / Ciberseguridad, Instituto Tecnológico de Las Américas (ITLA).